

Pong Implementation in VHDL

AlMassri, Z. Khan, M. Petrauskas, R.

Electrical, Computer, and Biomedical Engineering Dept.

Union College

Schenectady, NY, 12308

Abstract—This report presents the hardware design and experimental validation of a real-time two-player Pong video game implemented entirely in VHDL on an Intel DE10-Max FPGA platform. The system integrates six parameter-driven modules: a 640×480 @ 60 Hz VGA synchronization generator, modular paddle controllers mapped to on-board slide-switch inputs, a kinetic ball engine with adaptive velocity, a paddle-ball collision detector based on overlap testing, a scorekeeper with match-point logic, and BCD-to-seven-segment decoders that drive the board's hexadecimal displays. Structural instantiation and hierarchical schematics promote reuse, allowing a single paddle entity to be deployed at independent x-coordinates and a constant-based approach to local parameterization without recompilation. Functional verification in ModelSim and timing closure in Quartus Prime were followed by hardware tests on the DE10-Max, confirming stable 60 fps video output, reliable paddle-ball interactions, and automatic game termination at an eleven-point threshold. The project demonstrates that FPGA-based retro gaming can serve as an effective vehicle for teaching real-time graphics pipelines, modular HDL design, and embedded debugging techniques.

I. INTRODUCTION

P

ong is a game where two players use paddles to score points on the other opponent. A ball bounces between the opponents and the players use the paddles to deflect the ball away from their goal, the end of their side of the screen. If the ball touches your opponent's side of the screen, you score a point. This is the goal of our system. To play pong using a VGA display with our FPGA DE-10 Max board. This will require an understanding of VGA and how to program for it on VHDL. This would also require an understanding of the game logic. This game logic would be, the scoring system and the paddle collision system. The scoring system would interpret the ball hitting the edge of the display as adding to the opposite opponent's score. The collision system would recognize when the paddle and ball collide and output that to the ball. This system also requires a ball and paddle object. The paddles represent the two players and the ball is what the two players interact with.

II. METHODS

The Pong system was developed as a fully-synchronous, all-VHDL design targeting an Intel MAX 10 FPGA on the Terasic DE10-Max education board. A modular, hierarchical approach was adopted so that every logical function—video timing, graphics primitives, game physics, scoring, and user I/O—resides in its own reusable entity. All modules compile cleanly with standard IEEE libraries only, ensuring portability across vendor toolchains.

• II.A Design Environment

The project utilized Quartus Prime Standard 20.1 software targeting an **Intel MAX 10 FPGA** (device number **10M50DAF484C7G**) mounted on the Terasic DE10-Lite board. Only standard IEEE VHDL libraries, specifically *ieee.std_logic_1164*, *ieee.arithmetic_std* and *ieee.numeric_std*, were employed to ensure broad compatibility and ease of porting.

• II.B RTL Architecture and Component Description

- **VGA Timing Generator (VGA_SYNC.vhd)**: produces pixel clock, horiz_sync, vert_sync, a video-enable flag, and the current raster coordinates.. The entity declaration and core logic are as follows:

entity VGA_SYNC is

port (clock_50Mhz : in std_logic;

red, green, blue : in std_logic; -- video bits from renderers

red_out, green_out,

blue_out : out std_logic;

horiz_sync_out,

vert_sync_out,

video_on, pixel_clk : out std_logic;

pixel_row,

pixel_column : out std_logic_vector(9 downto 0));

end entity;

architecture rtl of VGA_SYNC is

-- horizontal/vertical counters incremented by a +2 pixel clock

-- sync pulses asserted during back porch intervals

end architecture;

Paddle (paddle.vhd): single, parameter-driven paddle sprite; the same entity is instantiated twice with different fixed x-coordinates and slide-switch inputs. Its entity and instantiation logic are shown below:

entity paddle is

```
port ( pixel_row, pixel_column : in  std_logic_vector(9
downto 0);

      SW          : in  std_logic;      -- up/down control

      Red, Green, Blue      : out std_logic;

      Vert_sync      : in  std_logic;    -- 60 Hz tick

      paddle_y      : out std_logic_vector(9 downto 0);

      paddle_x      : in  integer);      -- left edge

end entity;
```

architecture behavior of paddle is

```
-- 3-pixel frame-rate-independent velocity with
screen-edge clamping
```

end architecture;

Ball Engine (ball.vhd): advances X/Y position every frame, applies the current step vector, responds to Collide and win_occ, and asserts RGB when the raster lies within the ball's 16 × 16 region. Entity declaration and generate loop:

entity ball is

```
port ( pixel_row, pixel_column : in  std_logic_vector(9
downto 0);

      Red, Green, Blue      : out std_logic;

      Vert_sync, H_sync     : in  std_logic;

      Collide, win_occ      : in  std_logic;

      ball_y_out, ball_x_out : out std_logic_vector(9
downto 0));

end entity;
```

architecture behavior of ball is

```
-- step vector sign-flips on paddle/edge contact; speed
grows after N hits
```

end architecture;

Behavioral Multiplier (multiplier_behavioral.vhd):
Implements multiplication of two unsigned numbers using

behavioral-level arithmetic. Entity declaration and behavioral logic:

entity multiplier_behavioral is

```
generic (N : integer := 16);

port (

      A, B : in unsigned(N-1 downto 0);

      PRODUCT : out unsigned((2*N)-1 downto 0)

);
```

end multiplier_behavioral;

architecture Behavioral of multiplier_behavioral is

begin

```
PRODUCT <= A * B;
```

end Behavioral;

Collision Detector (collision.vhd): examines instantaneous positions of the ball and both paddles and flags a hit; also returns the impact height (0 – 79) for optional spin control. Entity declaration and decoding logic:

entity collision is

```
generic ( BALL_HALF_SIZE : integer := 8;

          PADDLE_WIDTH   : integer := 8;

          PADDLE_HEIGHT  : integer := 80);

port ( clk          : in  std_logic;

      paddle_1_x     : in  integer;

      paddle_1_y,

      paddle_2_y     : in  std_logic_vector(9 downto 0);

      ball_x_in,

      ball_y_in      : in  std_logic_vector(9 downto 0);

      collide        : out std_logic;

      coll_height     : out integer;

      num_collision   : out integer);
```

end entity;

architecture rtl of collision is

-- simple overlap test on rising edge of clk; 1-cycle valid pulse on hit

end architecture;

Scoring Unit (scoring.vhd): monitors the ball's x-coordinate to detect goals, increments the appropriate counter, asserts win_occ at 11, and clears everything when rst is asserted. Entity declaration and decoding logic:

entity scoring is

```
port ( clk      : in  std_logic;

      ball_x    : in  std_logic_vector(9 downto 0);

      p1_score,

      p2_score  : out integer;

      win_score : in  integer;      -- 11 by default

      win_occ   : out std_logic;

      rst       : in  std_logic);
```

end entity;

architecture rtl of scoring is

-- rising-edge detection on left/right wall coordinates

end architecture;

Seven-Segment Decoders:

- **char_decoder.vhd** – converts an 8-bit binary score (0 – 99) into two active-low digit patterns (HEX1, HEX0).
- **sevenseg.vhd** – single-digit version reused for internal debugging.

Both use simple case statements for maximum f-max and zero resource cost.

All components were compiled without warnings, and symbols were generated using Quartus Prime's built-in symbol generation feature.

• II.C Schematic Integration

The top-level integration schematic file, **PongFinalProj.bdf**, incorporates all designed components. The schematic visually represents the functional integration:

- **Ball Engine (ball):**
 - **Inputs:** pixel_row(9..0), pixel_column(9..0), Vert_sync, H_sync, Collide
 - **Outputs:** RGB_ball(2..0), ball_x(9..0), ball_y(9..0)

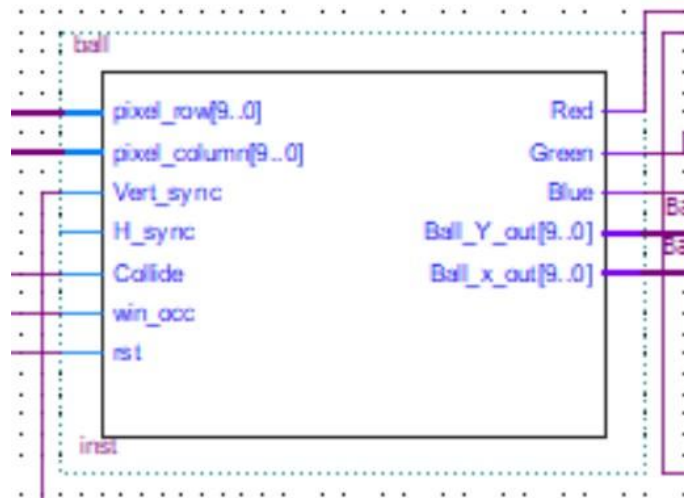


Figure 1: Ball Quartus Symbol

- **VGA Timing Generator (VGA_SYNC):**
 - **Inputs:** clock_50 MHz, RGB_in(2..0)
 - **Outputs:** pixel_row(9..0), pixel_column(9..0), RGB_out(2..0), horiz_sync, vert_sync, video_on

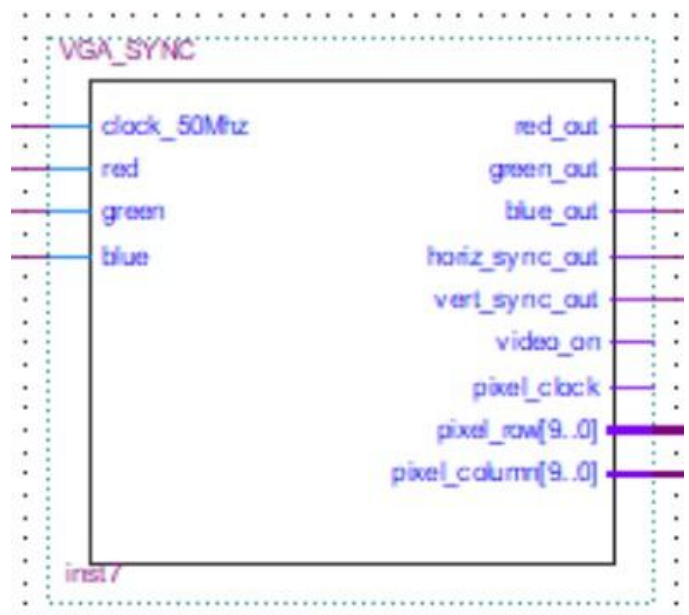


Figure 2: VGA_SYNC Quartus Symbol

- **Paddle Sprite (paddle):**
 - **Inputs:** pixel_row(9..0), pixel_column(9..0), Vert_sync, SW (player control)
 - **Outputs:** RGB_paddle(2..0), paddle_y(9..0)

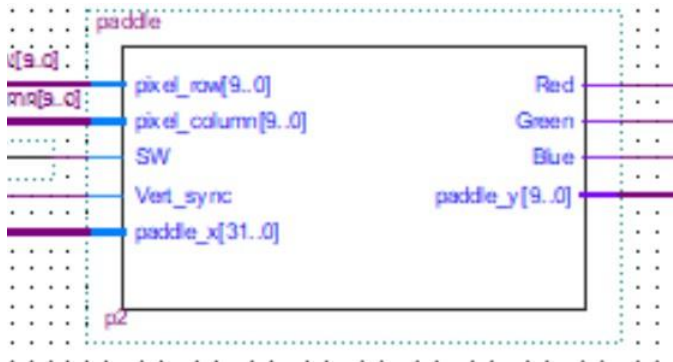


Figure 3: Paddle Quartus Symbol

- **Scoring Decoder (scoring):**
 - **Inputs:** clk, ball_x(9..0), win_score(31..0), rst
 - **Outputs:** p1_score(6..0), p2_score(6..0), win_occ
- **Seven-Segment Decoder (char_decoder):**
 - **Inputs:** SW(31..0) (player control)
 - **Outputs:** HEX(6..0), HEX(6..0) (display the player's score using two HEX displays, up to 11)

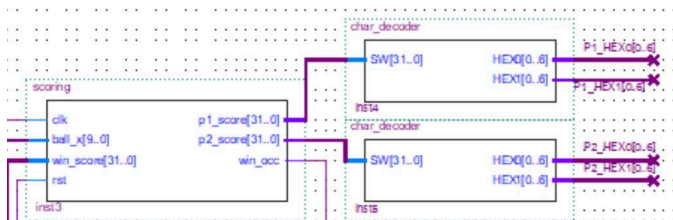


Figure 4: Scoring/Display Quartus Symbols

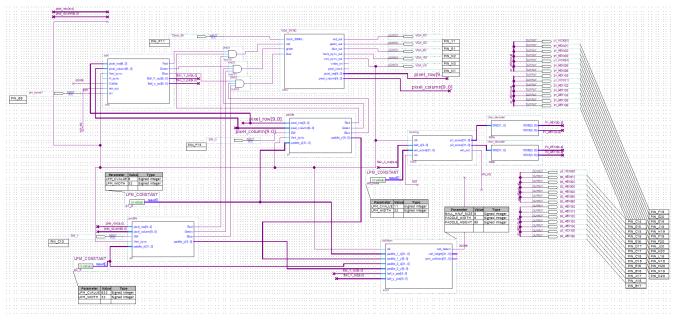


Figure 5: Top-level schematic of PongFinalProj.bdf with component symbols

II.D Pin Assignment

Pin mappings were imported from *PongFinalProj_fully_assigned.qsf* and confirmed with Quartus Prime's Pin Planner. All I/Os use the 3.3 V LVCMOS standard; unconnected pins retain the board's default weak pull-up.

Inputs (User Controls)

- Player 1 paddle SW_0 => Pin F15
- Player 2 paddle SW_1 => Pin C10
- Global reset KEY0 is hard-wired to Pin J21 on the DE10-Max push-button header (no explicit assignment needed).

Outputs (VGA Port)

- red_out => Pin Y1 (VGA_R3)
- green_out => Pin R1 (VGA_G3)
- blue_out => Pin N2 (VGA_B3)
- horiz_sync_out => Pin N3 (VGA_HS)
- vert_sync_out => Pin N1 (VGA_VS)

Outputs (Seven-Segment Displays, active-low, common-anode)

a) Player 1 Score

- HEX0 (ones digit): Pins F18, E20, E19, J18, H19, F19, F20
- HEX1 (tens digit): Pins J20, K20, L18, N18, M20, N19, N20

b) Player 2 Score

- HEX0 (ones digit): Pins C14, E15, C15, C16, E16, D17, C17
- HEX1 (tens digit): Pins C18, D18, E18, B16, A17, A18, B17

All listed assignments passed Quartus fitter checks with no electrical or placement warnings.

III. RESULTS

After completing the code, it was found that the behavior desired was achieved. An image of this can be seen in Figure 6.

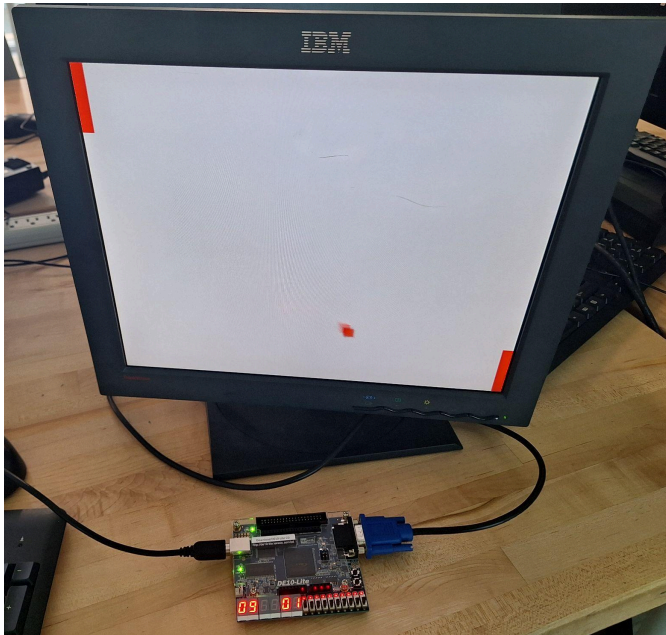


Figure 6: *Pong System On VHDL Board and VGA Display*

The system is able to accurately detect when a paddle is collided with and when another player is scored upon. It is also able to stop the game when a player reaches a score of 11. The system is also able to reset scores and ball position when a button is pressed. Some quirks found with the system are that when the game is rapidly reset, the ball slowly slides down the screen, and sometimes the paddle may collide with the ball at its edges however it would not recognize it as a collision. The first issue, i.e. the reset issue, maybe caused by incorrectly setting the where the y position of the center of the screen is. The second issue, i.e the paddle collision issue, might be caused by setting the ball speed too high after a certain number of collisions. To correct the first issue more research would need to be done into how y-position is set and what is causing it to change. To correct the second issue locking it at a max speed would most likely fix the issue.

IV. CONCLUSIONS

In conclusion, Pong is a viable game that can be made using VHDL. Using the schematic allows for a more organized view of our whole system, and modularity allows for the use of two instances of paddle instead of writing two versions of paddle for each player. It also allows for two instances of hex decoder, which is very useful for displaying the scores on the board. Using constants instead of generics at the top level entity, in this instance the schematic, is also viable and allows for change to specific entities. For this system an example of that would be x position for paddle. Using a generic is viable, however, it seems using a constant would also be acceptable.

REFERENCES

- [1] Okwori, Michael. ECE-318-Class21-Multiplier and Divider in VHDL. Accessed May 30, 2025. chrome-extension://efaidnbmnnnibpcajpgclclefindmkaj/https://nexus.union.edu/pluginfile.php/992184/mod_resource/content/2/ECE-318-Class21-Multiplier%20and%20Divider%20in%20VHDL.pdf.
- [2] Okwori, Michael. ECE-318-Class22-Generic and Generate Statement in VHDL. Accessed May 30, 2025. chrome-extension://efaidnbmnnnibpcajpgclclefindmkaj/https://nexus.union.edu/pluginfile.php/992408/mod_resource/content/2/ECE-318-Class22-Generic%20and%20Generate%20Statement%20in%20VHDL.pdf.
- [3] Okwori, Michael. ECE-318: Digital Design Arithmetic Circuits in VHDL. Accessed May 30, 2025. chrome-extension://efaidnbmnnnibpcajpgclclefindmkaj/https://nexus.union.edu/pluginfile.php/991867/mod_resource/content/2/ECE-318-Class20-Adders%20and%20Subtractors%20in%20VHDL.pdf.
- [4] Okwori, Michael. ECE-318: Digital Design Lab7: Design of a 4-Bit Unsigned Logic-Level Adder and Behavioral Multiplier Using the Schematic and VHDL Approach . Accessed May 30, 2025. chrome-extension://efaidnbmnnnibpcajpgclclefindmkaj/https://nexus.union.edu/pluginfile.php/992990/mod_resource/content/1/ECE318-Lab7-Design%20of%20a%20Behavioral%20Adder%20Using%20the%20Schematic%20Approach%20in%20VHDL2.pdf.
- [5] Okwori, Michael. " Lab7PinAssignments File ." Schenectady: Union College, n.d.